Reporte: Cinemática Diferencial de un Robot Planar de 3GDL

1. Declaración de Variables Simbólicas y Configuración del Robot

Se definen las variables simbólicas θ1(t), $\theta 2(t)$, $\theta 3(t)$ como funciones del tiempo, junto con las longitudes de eslabón l1,l2,l3. También se establece el vector RP = [0 0 0] que indica que las tres articulaciones son rotacionales (0 = revoluta, 1 = prismática).

En robótica, las coordenadas generalizadas son las variables que describen la configuración del robot. Para un robot planar de 3 grados de libertad con articulaciones de revolución, cada coordenada generalizada es un ángulo θi(t). Declarar estas variables como funciones del tiempo es necesario porque más adelante se derivarán respecto a *t* para obtener las velocidades generalizadas

```
%Limpieza de pantalla
clear all
close all
clc
%Declaración de variables simbólicas
syms th1(t) th2(t) th3(t) t l1 l2 l3
%Configuración del robot, 0 para junta rotacional, 1 para junta prismática
RP=[0 0 0];
%Creamos el vector de coordenadas articulares
Q= [th1, th2, th3];
disp('Coordenadas generalizadas');
```

```
Coordenadas generalizadas
```

```
pretty (Q);
```

```
(th1(t), th2(t), th3(t))
```

```
%Creamos el vector de velocidades generalizadas
Qp= diff(Q, t);
disp('Velocidades generalizadas');
```

```
Velocidades generalizadas
```

```
pretty (Qp);
```

```
/  d          d          d        \
| -- th1(t), -- th2(t), -- th3(t) |
\ dt         dt         dt        /
```

```
%Número de grado de libertad del robo t
GDL= size(RP,2);
GDL_str= num2str(GDL);
```

2. Posiciones Locales y Matrices de Rotación

Para cada eslabón i*i* se define:

- Un vector de posición local **P** que describe la posición del origen del marco {i} respecto al marco {i−1}.
- Una matriz de rotación local **R** que describe la orientación del marco {i} respecto al marco {i−1}.

$$ {}^{0}\mathbf{P}_1 = \begin{bmatrix} l_1 \cos\theta_1 \\ l_1 \sin\theta_1 \\ 0 \end{bmatrix} \quad {}^{0}\mathbf{R}_1 = \begin{bmatrix} \cos\theta_1 & -\sin\theta_1 & 0 \\ \sin\theta_1 & \cos\theta_1 & 0 \\ 0 & 0 & 1 \end{bmatrix} $$

Esto de bido a que en un robot planar, cada eslabón rota alrededor del eje z (perpendicular al plano de movimiento). La matriz de rotación corresponde a una rotación pura en z por un ángulo θi. El vector de posición indica que el extremo del eslabón i está a una distancia li del origen del marco anterior, en la dirección dada por el ángulo θi. El componente en z es cero porque el robot opera exclusivamente en el plano xy.

```
%Junta 1
%Posición de la junta 1 respecto a 0
P(:,:,1)= [l1*cos(th1); l1*sin(th1);0];
%Matriz de rotación de la junta 1 respecto a 0
R(:,:,1)= [cos(th1) -sin(th1) 0;
           sin(th1) cos(th1) 0;
           0            0    1];
%Junta 2
%Posición de la junta 2 respecto a 1
P(:,:,2)= [l2*cos(th2); l2*sin(th2);0];
%Matriz de rotación de la junta 2 respecto a 1
R(:,:,2)= [cos(th2) -sin(th2) 0;
           sin(th2) cos(th2) 0;
           0           0     1];

%junta 3
%Posición de la junta 3 respecto a 2
P(:,:,3)= [l3*cos(th3); l3*sin(th3);0];
%Matriz de rotación de la junta 3 respecto a 2
R(:,:,3)= [cos(th3) -sin(th3) 0;
           sin(th3) cos(th3) 0;
           0            0    1];
```

3. Matrices de Transformación Homogénea Locales (Ai)

Se construye la matriz de transformación homogénea local de cada eslabón combinando la rotación y la traslación, esto es necesario ya que la matriz de transformación homogénea es una herramienta que permite representar simultáneamente la rotación y la traslación entre dos marcos de referencia en una sola matriz de 4×4. La fila inferior [0 0 0 1] se añade para que la multiplicación de matrices encadene correctamente las transformaciones sucesivas. Cada Ai describe la relación local entre el marco {i−1} y el marco {i}.

4. Matrices de Transformación Homogénea Globales (Ti)

Se multiplican las matrices locales de forma acumulativa para obtener las transformaciones globales (respecto al marco base {0}):

$$T_1 = A_1, \quad T_2 = T_1 \cdot A_2, \quad T_3 = T_2 \cdot A_3$$

Esto es necesario ya que para conocer la posición y orientación de cualquier eslabón respecto al marco de referencia fijo, se necesita la composición de toda las transformaciones intermedias.

$$\begin{aligned}
&De\ \ la\ \ matriz\ \ T_i\ \ se\ \ extraen:\\
&R_i^0:\ \text{submatriz } 3\times 3\ \ \text{superior izquierda (rotación acumulada)}\\
&P_i^0:\ \text{vector columna de la cuarta columna (filas 1-3)}
\end{aligned}$$

$$^0\mathbf{P}_3 = \begin{bmatrix} l_1\cos\theta_1 + l_2\cos(\theta_1+\theta_2) + l_3\cos(\theta_1+\theta_2+\theta_3) \\ l_1\sin\theta_1 + l_2\sin(\theta_1+\theta_2) + l_3\sin(\theta_1+\theta_2+\theta_3) \\ 0 \end{bmatrix}$$

```
%Creamos un vector de ceros
Vector_Zeros= zeros(1, 3);
%Inicializamos las matrices de transformación Homogénea locales
A(:,:,GDL)=simplify([R(:,:,GDL) P(:,:,GDL); Vector_Zeros 1]);
%Inicializamos las matrices de transformación Homogénea globales
T(:,:,GDL)=simplify([R(:,:,GDL) P(:,:,GDL); Vector_Zeros 1]);

%Inicializamos las posiciones vistas desde el marco de referencia inercial
PO(:,:,GDL)= P(:,:,GDL);
%Inicializamos las matrices de rotación vistas desde el marco de referencia
inercial
RO(:,:,GDL)= R(:,:,GDL);
%Inicializamos las INVERSAS de las matrices de rotación vistas desde el
marco de referencia inercial
RO_inv(:,:,GDL)= R(:,:,GDL);
for i = 1:GDL
i_str= num2str(i);
%Locales
disp(strcat('Matriz de Transformación local A', i_str));
A(:,:,i)=simplify([R(:,:,i) P(:,:,i); Vector_Zeros 1]);
pretty (A(:,:,i));
%Globales
try
T(:,:,i)= T(:,:,i-1)*A(:,:,i);
catch
T(:,:,i)= A(:,:,i);
end
disp(strcat('Matriz de Transformación global T', i_str));
```

```
T(:,:,i)= simplify(T(:,:,i));
pretty(T(:,:,i))
RO(:,:,i)= T(1:3,1:3,i);
RO_inv(:,:,i)= transpose(RO(:,:,i));
PO(:,:,i)= T(1:3,4,i);
end
```

```
Matriz de Transformación local A1
/ cos(th1(t)), -sin(th1(t)), 0, l1 cos(th1(t)) \
|                                              |
| sin(th1(t)),  cos(th1(t)), 0, l1 sin(th1(t)) |
|                                              |
|      0,            0,      1,        0       |
|                                              |
\      0,            0,      0,        1       /
Matriz de Transformación global T1
/ cos(th1(t)), -sin(th1(t)), 0, l1 cos(th1(t)) \
|                                              |
| sin(th1(t)),  cos(th1(t)), 0, l1 sin(th1(t)) |
|                                              |
|      0,            0,      1,        0       |
|                                              |
\      0,            0,      0,        1       /
Matriz de Transformación local A2
/ cos(th2(t)), -sin(th2(t)), 0, l2 cos(th2(t)) \
|                                              |
| sin(th2(t)),  cos(th2(t)), 0, l2 sin(th2(t)) |
|                                              |
|      0,            0,      1,        0       |
|                                              |
\      0,            0,      0,        1       /
Matriz de Transformación global T2
/ #2, -#1, 0, l1 cos(th1(t)) + l2 #2 \
|                                    |
| #1,  #2, 0, l1 sin(th1(t)) + l2 #1 |
|                                    |
|  0,  0,  1,            0           |
|                                    |
\  0,  0,  0,            1           /

where

   #1 == sin(th1(t) + th2(t))

   #2 == cos(th1(t) + th2(t))
Matriz de Transformación local A3
/ cos(th3(t)), -sin(th3(t)), 0, l3 cos(th3(t)) \
|                                              |
| sin(th3(t)),  cos(th3(t)), 0, l3 sin(th3(t)) |
|                                              |
|      0,            0,      1,        0       |
|                                              |
\      0,            0,      0,        1       /
Matriz de Transformación global T3
/ #2, -#1, 0, l1 cos(th1(t)) + l3 #2 + l2 cos(th1(t) + th2(t)) \
|                                                               |
| #1,  #2, 0, l1 sin(th1(t)) + l3 #1 + l2 sin(th1(t) + th2(t)) |
|                                                               |
|  0,  0,  1,                          0                        |
|                                                               |
\  0,  0,  0,                          1                        /

where
```

```
#1 == sin(th1(t) + th2(t) + th3(t))

#2 == cos(th1(t) + th2(t) + th3(t))
```

5. Jacobiano Lineal por el Método Diferencial

Se calcula el Jacobiano analítico tomando las derivadas parciales de la posición del efector final respecto a cada coordenada articular. Cada columna *j* del Jacobiano indica cómo cambia la posición del efector final cuando se varía únicamente la articulación *j*. Este método es conceptualmente directo: se derivan las ecuaciones de posición directamente.

En MATLAB se utiliza `functionalDerivative` para calcular cada derivada parcial. El resultado para el robot planar 3R es:

$$J_v^{\text{dif}} = \begin{bmatrix} -l_1 s_1 - l_2 s_{12} - l_3 s_{123} & -l_2 s_{12} - l_3 s_{123} & -l_3 s_{123} \\ l_1 c_1 + l_2 c_{12} + l_3 c_{123} & l_2 c_{12} + l_3 c_{123} & l_3 c_{123} \\ 0 & 0 & 0 \end{bmatrix}$$

$$\text{donde } s_{12} = \sin(\theta_1 + \theta_2),\ c_{123} = \cos(\theta_1 + \theta_2 + \theta_3).$$

```
disp('Jacobiano lineal obtenido de forma diferencial');
```

```
Jacobiano lineal obtenido de forma diferencial
```

```
%Derivadas parciales de x respecto a th1 y th2
Jv11= functionalDerivative(PO(1,1,GDL), th1);
Jv12= functionalDerivative(PO(1,1,GDL), th2);
Jv13= functionalDerivative(PO(1,1,GDL), th3);
%Derivadas parciales de y respecto a th1 y th2
Jv21= functionalDerivative(PO(2,1,GDL), th1);
Jv22= functionalDerivative(PO(2,1,GDL), th2);
Jv23= functionalDerivative(PO(2,1,GDL), th3);
%Derivadas parciales de z respecto a th1 y th2
Jv31= functionalDerivative(PO(3,1,GDL), th1);
Jv32= functionalDerivative(PO(3,1,GDL), th2);
Jv33= functionalDerivative(PO(3,1,GDL), th3);
%Creamos la matríz del Jacobiano lineal
jv_d=simplify([Jv11 Jv12 Jv13;
               Jv21 Jv22 Jv23;
               Jv31 Jv32 Jv33]);
pretty(jv_d);
```

```
/ - l1 sin(th1(t)) - #1 - l2 sin(th1(t) + th2(t)), - #1 - l2 sin(th1(t) + th2(t)), -#1 \
|                                                                                     |
|  l1 cos(th1(t)) + #2 + l2 cos(th1(t) + th2(t)),   #2 + l2 cos(th1(t) + th2(t)),   #2 |
|                                                                                     |
\                        0,                                        0,                0  /

where
```

```
#1 == l3 sin(th1(t) + th2(t) + th3(t))

#2 == l3 cos(th1(t) + th2(t) + th3(t))
```

6. Jacobiano Lineal por el Método Análitico

Se calcula el Jacobiano lineal usando la fórmula geométrica. Para cada articulación rotacional i. Esta formulación proviene de la relación entre velocidad lineal y angular de un cuerpo rígido: **v**=***ω***×**r**

Cuando la articulación i gira con velocidad $\dot{\theta}_i$, genera una velocidad angular $\dot{\theta}_i \hat{z}_{i-1}$ .La contribución a la velocidad lineal del efector final es el producto cruz de esa velocidad angular con el vector de distancia desde la articulación hasta el efector final. Esto es exactamente lo que calcula la fórmula del Jacobiano geométrico para juntas de revolución.

El resultado coincide exactamente con el obtenido por derivadas parciales, lo que valida ambos métodos.

7. Jacobiano Angular

Para cada articulación rotacional i, la columna del Jacobiano angular es simplemente el eje de rotación de esa articulación visto desde el marco base:

$$J_{\omega,i} = \hat{z}_{i-1}$$

La velocidad angular del efector final se compone de la suma de las contribuciones angulares de cada articulación. Para una articulación de revolución, la contribución angular es la velocidad de la articulación multiplicada por su eje de rotación. En un robot planar, todas las articulaciones rotan alrededor del eje *zz* del plano, por lo que:

$$J_\omega = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Esto indica que cada articulación contribuye únicamente a la velocidad angular en z. Si el robot tuviera articulaciones prismáticas, las columnas correspondientes del Jacobiano angular serían vectores de ceros, porque una articulación prismática no genera rotación; solo permite el movimiento de traslación lineal.

```
%Calculamos el jacobiano lineal de forma analítica
Jv_a(:,GDL)=PO(:,:,GDL);
Jw_a(:,GDL)=PO(:,:,GDL);
for k= 1:GDL
if RP(k)==0 %Casos: articulación rotacional
%Para las juntas de revolución
try
Jv_a(:,k)= cross(RO(:,3,k-1), PO(:,:,GDL)-PO(:,:,k-1));
```

```
Jw_a(:,k)= RO(:,3,k-1);
catch
Jv_a(:,k)= cross([0,0,1], PO(:,:,GDL));
Jw_a(:,k)=[0,0,1];
end
%Para las juntas prismáticas
elseif RP(k)==1 %Casos: articulación prismática
%
try
Jv_a(:,k)= RO(:,3,k-1);
catch
Jv_a(:,k)=[0,0,1];
end
Jw_a(:,k)=[0,0,0];
end
end
Jv_a= simplify (Jv_a);
Jw_a= simplify (Jw_a);
disp('Jacobiano lineal obtenido de forma analítica');
```

```
Jacobiano lineal obtenido de forma analítica
```

```
pretty (Jv_a);
```

```
/ - l1 sin(th1(t)) - #1 - l2 sin(th1(t) + th2(t)), - #1 - l2 sin(th1(t) + th2(t)), -#1 \
|                                                                                     |
|  l1 cos(th1(t)) + #2 + l2 cos(th1(t) + th2(t)),   #2 + l2 cos(th1(t) + th2(t)),   #2 |
|                                                                                     |
\                        0,                                        0,                0  /

where

   #1 == l3 sin(th1(t) + th2(t) + th3(t))

   #2 == l3 cos(th1(t) + th2(t) + th3(t))
```

```
disp('Jacobiano ángular obtenido de forma analítica');
```

```
Jacobiano ángular obtenido de forma analítica
```

```
pretty (Jw_a);
```

```
/ 0, 0, 0 \
|         |
| 0, 0, 0 |
|         |
\ 1, 1, 1 /
```

8. Velocidad Lineal del Efector Final

Se multiplica el Jacobiano lineal por el vector de velocidades articulares:

```
disp('Velocidad lineal obtenida mediante el Jacobiano lineal');
```

```
Velocidad lineal obtenida mediante el Jacobiano lineal
```

```
V=simplify (Jv_a*Qp');
pretty(V);
```

```
/ - #4 (l3 sin(#1) + l2 sin(#2)) - #5 (l1 sin(th1(t)) + l3 sin(#1) + l2 sin(#2)) - l3 #3 sin(#1) \
|                                                                                                |
|  #4 (l3 cos(#1) + l2 cos(#2)) + #5 (l1 cos(th1(t)) + l3 cos(#1) + l2 cos(#2)) + l3 #3 cos(#1)  |
|                                                                                                |
\                                                 0                                              /

where

   #1 == th1(t) + th2(t) + th3(t)

   #2 == th1(t) + th2(t)

         _________
          d
   #3 == -- th3(t)
         dt

         _________
          d
   #4 == -- th2(t)
         dt

         _________
          d
   #5 == -- th1(t)
         dt
```

9. Velocidad Angular del Efector Final

Se multiplica el Jacobiano angular por el vector de velocidades articulares:

```
disp('Velocidad angular obtenida mediante el Jacobiano angular');
```

```
Velocidad angular obtenida mediante el Jacobiano angular
```

```
W=simplify (Jw_a*Qp');
pretty(W);
```

```
/                 0                 \
|                                   |
|                 0                 |
|                                   |
| _________   _________   _________ |
|  d           d           d        |
| -- th1(t) + -- th2(t) + -- th3(t) |
\ dt          dt          dt        /
```

Conclusión

Este resultado es la ecuación de cinemática diferencial del robot, que es fundamental para planeación de trayectorias, análisis de singularidades, control de fuerza, y cálculo de la cinemática inversa diferencial. Ambos métodos producen el mismo Jacobiano lineal, lo cual valida la correctitud del código.